

OPERATING SYSTEM

PROJECT 0 – System Call

1. General rule

- The project is done in groups: each group has a maximum of **3 students**.
- **The same exercises will all be scored 0 for the entire practice (even though there are scores for other exercises and practice projects).**

2. Submission

Submit assignments directly on the course website (MOODLE), not accepting submissions via email or other forms.

Filename: **StudentID1_StudentID2_StudentID3.zip** (with StudentID1 < StudentID2 < StudentID3).

Ex: Your group has 3 students: 2312001, 2312002 and 2312003, the filename is:
2312001_2312002_2312003.zip.

Include:

- **StudentID1_StudentID2_StudentID3_Report.pdf:** Writeups should be short and sweet. Do not spend too much effort or include your source code on your writeups. The purpose of the report is to give you an opportunity to clarify your understandings, any problems with your work, and to add information that may be useful in grading. If you had specific problems or issues, approaches you tried that didn't work, or concepts that were not fully understood, then an explanation in your report may help us to assign partial credit.
- **Release:** File diff (diff patch, Ex: \$ git diff > **StudentID1_StudentID2_StudentID3.patch**).
- **Source:** Zip file of xv6 (the version is "made clean").

3. Demo Interviews

Your work is graded on code understanding.

When administering this course, we do our best to give a fair assessment to *each individual* based on each person's understanding/contribution to the project.

4. How to add a new System call in xv6:

Suppose you want to create a system call named `hello`, here are the step by step details:

In Kernel space

1) In `syscall.h`, define a new system call number (usually increased by 1 from the last one).

```
#define SYS_hello 22
```

2) In `syscall.c`, define the prototype for the function that handles the system call and map the previous defined system call number to that function.

```
extern int sys_hello(void);
static int (*syscalls[]) (void) = {
    ...
    [SYS_hello] = sys_hello,
};
```

3) In `sysproc.c`, implement for the system call function.

```
int sys_hello(void) {
    printf("Hello, world!\n");
    return 0;
}
```

In User space

4) In `user.h`, define the function that the user can use to invoke the new implemented system call.

```
int hello(void);
```

5) In `usys.pl`, add an interface for the user function to access the system call.

```
entry("hello");
```

6) Write a user program called `testhello.c` to test the new system call.

```
#include "user.h"

int main() {
    hello();
    exit(0);
}
```

Compilation and test

7) Add to Makefile for compilation:

```
$U/_testhello\
```

8) Build and run a user program:

```
$ make qemu
$ ./testhello
```

5. How the system call deal with the arguments passed from the user program or return data back to user program in xv6:

See `sys_fstat()`, `sys_exec()` (`kernel/sysfile.c`) and `filestat()` (`kernel/file.c`) for examples.

- Use `argint()`, `argaddr()`, or `fetchaddr()` (which calls `copyin()`) or other relates to get arguments passed from the user program.
- Use `copyout()` to return data back to user program.

6. Requirements

In this lab you will add some new system calls to xv6, which will help you understand how they work and will expose you to some of the internals of the xv6 kernel. You will add more system calls in later labs.

Before you start coding, read Chapter 2 of the [xv6 book](#), and Sections 4.3 and 4.4 of Chapter 4, and related source files:

- The user-space "stubs" that route system calls into the kernel are in `user/usys.S`, which is generated by `user/usys.pl` when you run `make`. Declarations are in `user/user.h`
- The kernel-space code that routes a system call to the kernel function that implements it is in `kernel/syscall.c` and `kernel/syscall.h`.
- Process-related code is `kernel/proc.h` and `kernel/proc.c`.

To start the lab, switch to the `syscall` branch:

```
$ git fetch
$ git checkout syscall
$ make clean
```

Trace system call

In this assignment you will add a system call tracing feature that may help you when debugging later labs. You'll create a new `trace` system call that will control tracing. It should take one argument, an integer "mask", whose bits specify which system calls to trace. For example, to trace the `read` system call, a program calls `trace(1 << SYS_read)`, where `SYS_read` is a syscall number from `kernel/syscall.h`. You have to modify the xv6 kernel to print out a line when each system call is about to return, if the system call's number is set in the mask. The line should contain the name of the system call and the return value. The `trace` system call should enable tracing for the process that calls it, but should not affect other processes.

Write a `trace` user-level program that runs another program with tracing enabled (`user/trace.c`). When you're done, you should see output like this

```
$ trace 32 grep hello README
syscall read -> 1023
syscall read -> 966
syscall read -> 70
syscall read -> 0
$
$ trace 2147483647 grep hello README
syscall trace -> 0
syscall exec -> 3
syscall open -> 3
syscall read -> 1023
syscall read -> 966
syscall read -> 70
syscall read -> 0
syscall close -> 0
$
$ grep hello README
$
```

In the first example above, `trace` invokes `grep` tracing just the `read` system call. The 32 is `1<<SYS_read`. In the second example, `trace` runs `grep` while tracing all system calls; the 2147483647 has all 31 low bits set. In the third example, the program isn't traced, so no trace output is printed. Your solution is correct if your program behaves as shown.

Some hints:

- Write the user program `user/trace.c` that will take the first argument (`argv[1]`) as the mask and the remaining arguments (`argv[2]`, `argv[3]`,...) are the program (`argv[2]`) with its arguments (`argv[3]`,...) to be executed and traced. This user program will call the `trace` system call with the mask before calling `exec` system call to execute and trace the user program named `argv[2]`.
- Add `$U/_trace` to `UPROGS` in `Makefile`
- Run `make qemu` and you will see that the compiler cannot compile `user/trace.c`, because the user-space stubs for the system call don't exist yet: add a prototype for the system call to `user/user.h`, a stub to `user/usys.pl`, and a syscall number to `kernel/syscall.h`. The `Makefile` invokes the perl script `user/usys.pl`, which produces `user/usys.S`, the actual system call stubs, which use the RISC-V `ecall` instruction to transition to the kernel. Once you fix the compilation issues, run `trace 32 grep hello README`; it will fail because you haven't implemented the system call in the kernel yet.
- Add a `sys_trace()` function in `kernel/sysproc.c` that implements the new system call by remembering its first argument (the mask above) in a new added variable in the `proc` structure in `kernel/proc.h`. The functions to retrieve system call arguments from user space are in `kernel/syscall.c`, and you can see examples of their use in `kernel/sysproc.c` (we use `argint()` this case as the mask is integer).
- Modify the `syscall()` function in `kernel/syscall.c` to print the trace output. You will need to add an array of syscall names to index into.

7. Grade

No.	Exercise	Grade
1	trace	10

8. Reference

- <https://pdos.csail.mit.edu/6.1810/2023/labs/syscall.html>